EE627 — Class Summary

March 26, 2026

Prof. Rensheng Wang

Contents

1 Midterm Structure and Philosophy **2**
- 1.1 Format and Timeline . . . 2
- 1.2 Data and Scope . . . 2
- 1.3 What Is Actually Being Tested . . . 2

2 Midterm Part 1 — Extended Feature Engineering **2**
- 2.1 Background: Homework 4 Baseline . . . 2
- 2.2 Part 1A: Statistical Genre Features . . . 2
- 2.3 Part 1B: Decision Logic for Ranking Score (Open-Ended) . . . 3
- 2.4 Philosophy: Feature Expansion Before Feature Selection . . . 3

3 Midterm Part 2 — Cold Start Problem **3**
- 3.1 What Is the Cold Start Problem? . . . 3
- 3.2 Strategy 2A: Global Popularity Fallback . . . 3
- 3.3 Strategy 2B: Sibling Track Inference . . . 3
- 3.4 Implementation Note . . . 4

4 Feature Selection **4**
- 4.1 Motivation . . . 4
- 4.2 Method 1: Filter-Based Selection . . . 4
- 4.3 Method 2: Wrapper-Based Selection (Recursive Feature Elimination) . . . 5
  - 4.3.1 Why Individual Ranking Is Insufficient . . . 5
  - 4.3.2 Forward Selection (Naïve — Not Recommended for Interviews) . . . 5
  - 4.3.3 Recursive Feature Elimination (RFE) — Industry Standard . . . 5
- 4.4 Method 3: Embedded Selection . . . 5
  - 4.4.1 Random Forest . . . 6
  - 4.4.2 Lasso Regularization (L1) . . . 6
- 4.5 Comparison of Feature Selection Methods . . . 6

5 Industry Context: ML Engineers in the AI Era **6**
- 5.1 Shift in Job Market Demand . . . 6
- 5.2 New Interview Standard . . . 6
- 5.3 Practical Takeaway . . . 7

6 Key Formulas Reference **7**

1 Midterm Structure and Philosophy

1.1 Format and Timeline

- The midterm will be posted by the weekend of March 28–29, 2026.
- Students will have **more than 7 days** to complete it — treat it like a regular homework with an extended window.
- **Missing the deadline results in zero credit.** No exceptions.
- The midterm is **open-book and AI-assisted**; students are encouraged to use ChatGPT, Google Gemini, GitHub Copilot, etc.

1.2 Data and Scope

The midterm **reuses the Yahoo Music recommendation dataset** from Homework 4. No new data is introduced so students can focus on extending prior work rather than re-learning a new domain.

1.3 What Is Actually Being Tested

The professor explicitly moved away from closed, ground-truth questions ("answer is 2", "answer is ABCD"). Instead, the midterm tests three compounding skills:

1. **Conceptual depth** — Can you understand *why* a method works, not just copy code?
2. **AI collaboration** — Can you guide an AI tool iteratively, challenge its output, and synthesize a better result?
3. **Analytical initiative** — Given an open-ended problem, do you expand the solution space or stop at the minimum?

Key analogy from lecture: A junior engineer stops at "AI gave me an answer." A skilled engineer uses AI as a multiplier — starting from their own knowledge base and driving the conversation deeper. The professor estimated this effect as $1 \times \text{AI} = 10\text{x}$–$100\text{x}$ output quality.

2 Midterm Part 1 — Extended Feature Engineering

2.1 Background: Homework 4 Baseline

In Homework 4, students built a user–track scoring table with three score dimensions:

- **Album (urban) score** — unique rating for a user–album pair.
- **Artist score** — unique rating for a user–artist pair.
- **Genre score** — *not* unique; one track can have 3, 5, or 7 genres.

2.2 Part 1A: Statistical Genre Features

Because genre is a multi-valued attribute, a scalar genre score cannot be assigned directly. Instead, derive **summary statistics** across all genres associated with a track:

$$\text{Genre features} = \{\mu,\ \sigma,\ \text{min},\ \text{max},\ \text{median},\ \text{variance},\ \ldots\}$$

This converts the variable-length genre list into a fixed-width feature vector.

2.3 Part 1B: Decision Logic for Ranking Score (Open-Ended)

Given album score, artist score, and genre statistics, students must define their own **composite ranking score**. Possible strategies include:

- Simple sum: score $= s_{\text{album}} + s_{\text{artist}}$
- Maximum: score $= \max(s_{\text{album}},\ s_{\text{artist}})$
- Weighted combination: score $= w_1 s_{\text{album}} + w_2 s_{\text{artist}} + w_3 \bar{s}_{\text{genre}}$
- Any other defensible aggregation of multiple dimensions.

Grading note: More diversified and justified strategies earn more credit. Simply resubmitting Homework 4 results earns near-zero credit.

2.4 Philosophy: Feature Expansion Before Feature Selection

Add *as many measurement dimensions as possible* first. Redundant or unhelpful features can always be removed later (see Section 4). The reverse — trying to recover information you never collected — is impossible.

3 Midterm Part 2 — Cold Start Problem

3.1 What Is the Cold Start Problem?

A **cold start** occurs when a test user has no matching history in the training data: all album, artist, and genre lookups return empty. Without any personalized signal, standard history-based scoring fails entirely.

3.2 Strategy 2A: Global Popularity Fallback

Idea: When user history is absent, fall back to *population-level* statistics.

1. Compute a global average (or rank) score for each track across *all* users and all historical ratings.
2. For a cold-start user, rank the candidate tracks by their global popularity score.
3. This produces a non-personalized recommendation that is still better than random.

Real-world analogy: When a brand-new Netflix user registers, Netflix recommends its most-watched or highest-rated content globally until it learns the user's preferences.

3.3 Strategy 2B: Sibling Track Inference

Idea: Even if a user has never rated album A, they may have rated *other tracks in album A*. These "sibling tracks" provide a proxy signal.

1. For target track t in album A, find all other tracks $\{t_1, t_2, \ldots, t_k\}$ in album A.
2. Check whether the cold-start user has rating history for any t_i.
3. If ratings exist, compute statistical features:

$$\text{sibling features} = \{\mu_{\text{sibling}},\ \sigma_{\text{sibling}},\ \text{min},\ \text{max},\ \text{etc.}\}$$

4. Use these statistics as a surrogate album score to rank track t.

Real-world analogy (from lecture): An artist releases an album (e.g., Lady Gaga's *The Fame*). If a user has highly rated several songs from *The Fame*, it is reasonable to infer they would also enjoy *Poker Face*, even if they have not explicitly rated it.

3.4 Implementation Note

Both strategies can be implemented without writing code from scratch. Providing the data context and strategy description to an AI coding assistant (Gemini, ChatGPT) is sufficient to generate working Python. The student's role is to **design the strategy and evaluate the results**, not to debug syntax.

4 Feature Selection

4.1 Motivation

Adding features is cheap; keeping all of them is costly:

- Too many features relative to samples $\Rightarrow$ **overfitting**.
- Duplicate/correlated features introduce **noise dimensions** that the model cannot ignore.
- When #features > #samples, the model learns noise as signal $\Rightarrow$ **high variance, low generalization**.
- Parsimony principle: *given equal performance, prefer the simpler model.*

"Garbage in, garbage out" — feature selection filters uninformative features so only meaningful signal reaches the model.

4.2 Method 1: Filter-Based Selection

Evaluate each feature independently against the target variable.

1. Compute a relevance metric for every feature x_j with respect to y (e.g., Pearson correlation $\rho(x_j, y)$).
2. Rank features by their metric score.
3. Keep the top-k features; discard the rest.

NBA analogy from lecture: Evaluate each player individually in a tryout; keep the top scorers.

Python example:

```
import numpy as np
corr = [np.corrcoef(X[:, j], y)[0, 1] for j in range(X.shape[1])]
top_k_indices = np.argsort(np.abs(corr))[::-1][:k]
```

4.3 Method 2: Wrapper-Based Selection (Recursive Feature Elimination)

4.3.1 Why Individual Ranking Is Insufficient

Features interact — the combined performance of a subset is *not* the sum of individual performances:

$$\text{Benefit}(A + B) \neq \text{Benefit}(A) + \text{Benefit}(B)$$

Therefore, subsets must be evaluated *as units*.

4.3.2 Forward Selection (Naïve — Not Recommended for Interviews)

1. Select the single best feature.
2. Greedily add the feature that most improves the subset score.
3. Repeat until the desired number of features is reached.

4.3.3 Recursive Feature Elimination (RFE) — Industry Standard

Preferred by financial firms (Two Sigma, Goldman Sachs) in pre-screening assessments.

1. Start with all n features.
2. For $i = 1$ to n: drop feature i temporarily and evaluate model performance on remaining $n - 1$ features.
3. Permanently drop the feature whose removal had the *least negative impact* (i.e., was most redundant).
4. Repeat with $n - 1$ features until the desired feature count is reached.

Corporate analogy: A company performing layoffs does not identify the best employee to retain — it identifies the *lowest performer* each round and eliminates them.

Python example (scikit-learn):

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression

model = LinearRegression()
selector = RFE(model, n_features_to_select=10)
selector.fit(X_train, y_train)
X_reduced = selector.transform(X_train)
```

4.4 Method 3: Embedded Selection

Feature selection is **built into the learning algorithm itself** — no separate step required.

4.4.1 Random Forest

- Each tree is built from a *random subset* of features (e.g., 30% of columns).
- Features that never improve splits receive zero importance and are implicitly excluded.
- `feature_importances_` attribute ranks features by contribution.

4.4.2 Lasso Regularization (L1)

Lasso adds an L1 penalty to the linear regression loss:

$$\mathcal{L}(\mathbf{w}) = \underbrace{\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2}_{\text{regression loss}} + \lambda \underbrace{\|\mathbf{w}\|_1}_{\text{L1 penalty}}$$

- The L1 penalty geometrically encourages solutions where many $w_j = 0$.
- When $w_j = 0$, feature x_j is effectively removed from the model.
- λ controls sparsity: larger $\lambda \Rightarrow$ more features zeroed out.
- Unlike Ridge (L2), Lasso produces *exact* zeros, making it a true embedded selector.

Python example:

```
from sklearn.linear_model import Lasso
model = Lasso(alpha=0.01)
model.fit(X_train, y_train)
selected = np.where(model.coef_ != 0)[0]
```

4.5 Comparison of Feature Selection Methods

Method	Evaluation Unit	Complexity	Typical Use Case
Filter-based	Individual feature	Low	Quick baseline, EDA
Wrapper-based (RFE)	Feature subset	High	Finance pre-screening, ML interviews
Embedded (Lasso)	Model weights	Medium	Linear models, sparsity
Embedded (RF)	Tree splits	Medium–High	Non-linear, tabular data

5 Industry Context: ML Engineers in the AI Era

5.1 Shift in Job Market Demand

- Pure software development roles (front-end, back-end) are increasingly automatable by AI.
- **Machine learning / AI engineer** roles are growing in market share.
- Companies now expect engineers to *direct* AI, not simply write code.

5.2 New Interview Standard

Modern technical interviews (including AI-assisted rounds at major companies) test:

1. Whether the candidate can **challenge and refine** AI output rather than accept it blindly.
2. Whether the candidate can **ask the right follow-up questions** to deepen an AI-generated analysis.
3. Whether the candidate can make **strategic judgments** (feature selection, model choice, evaluation criteria) that an AI cannot make without domain guidance.

5.3 Practical Takeaway

- Use AI for low-level implementation (code generation, error fixing, report formatting).
- Reserve human judgment for high-level strategy: *what features to engineer, which model to select, how to evaluate and iterate.*
- A student who only copies AI output learns nothing and will be screened out in interviews.
- A student who uses AI as a *force multiplier* for their own thinking produces work of dramatically higher quality.

6 Key Formulas Reference

Pearson Correlation (Filter-Based): $\rho(X_j, Y) = \dfrac{\mathrm{Cov}(X_j, Y)}{\sigma_{X_j}\,\sigma_Y}$

Linear Regression Loss: $\mathcal{L} = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$

Lasso (L1) Regularized Loss: $\mathcal{L}_{\text{Lasso}} = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_1$

Ridge (L2) Regularized Loss (preview): $\mathcal{L}_{\text{Ridge}} = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2$